Intermediate DevOps
Day 1: Continuous integration

United Airlines · Pluralsight instructor-led training

Kangeyan Passoubady
9-11 November 2026

KavinSchool

Prerequisites

- Basic IT and software development experience.
- Some scripting or programming experience.
- Exposure to basic DevOps principles.
- Comfort working at a Linux command line.

Recommended preparation: Pluralsight DevOps Foundations Skill IQ and about 30 minutes of its recommended learning.

CHAT WATERFALL

Introduce Yourself

Draft one short chat message:

1. Your name and location or time zone.
2. Your role.
3. Your experience in IT and at your current organization.

Type without sending. Send together after the countdown: 3... 2... 1... send.

You may speak when invited, keep a private note, or pass. Share only what feels comfortable.

A Little Beyond Work

Choose one prompt. Share only what feels comfortable.

01

Bucket List

Name one to three experiences you hope to have.

02

Recent Trip

Share a place you visited and one detail that stayed with you.

03

Memorable Moment

Share a recent moment that made you proud, grateful, or happy.

60 seconds: Share in chat, speak when invited, keep a private note, or pass.

CHAT WATERFALL

Your DevOps Experience

Draft one short response:

1. Your starting point with Git, pipelines, Terraform, Docker or Kubernetes.
2. Your main work focus: development, analysis, testing, operations or another area.
3. One delivery problem or DevOps question you want this course to address.

Type without sending. Send together after the countdown: 3... 2... 1... send.

Speaking when invited, a private note, and passing are equally welcome.

Press ↓ for supporting detail: What We Learned (key points)

KEY POINTS

REFERENCE ONLY

DEEP DIVE / KEY POINTS

Your DevOps Experience: What We Learned

- Starting points: Day 1 builds from Git changes to automated tests in Harness.
- Work focus: development and testing connect to infrastructure and credentials on Day 2, then deployment and operations on Day 3.
- Key questions: use pipeline logs, Terraform plans and rollout health to explain failures and choose a next step.

Compare these themes with your response. Note one lab or topic to revisit.

About this course

- Instructor: Kangeyan Passoubady (Kangs).
- About 30% of the time is lecture. About 70% is hands-on lab work.
- Each lecture prepares you for the lab that follows it.
- Ask questions at any time, by voice or in the chat.

The three days

Day	Lectures	Labs
1 · Continuous integration	DevOps foundations, version control and CI, pipelines in Harness, testing and quality gates	01, 02, 03
2 · Infrastructure as code (IaC) and credentials	IaC concepts, Terraform, IaC in pipelines, credential management	04, 05, 06
3 · Containers, Kubernetes and delivery	Containers, Elastic Container Service (ECS) and Fargate, Kubernetes, continuous delivery	07, 08, 09

The labs build on each other. The application and pipelines from Day 1 are used again on Days 2 and 3.

Day 1 agenda

MORNING SESSIONS — AM

1	KICKOFF	Welcome and course overview
2	LECTURE	DevOps foundations
3	LECTURE	Version control & continuous integration
4	LAB 01	Lab environment and Harness Open Source
5	LECTURE	Pipelines and Harness

AFTERNOON SESSIONS — PM

6	LAB 02	Your first CI pipeline
7	LECTURE	Testing and quality gates
8	LAB 03	Quality gates, PRs & integration tests
9	WRAP-UP	Summary and questions

How the labs work

- The labs are Markdown files in the course repository: `https://github.com/kpassoubady/intermediate-devops`.
- Each step explains why before it shows a command. Read the explanation before you run anything.
- Checkpoints show what you should see. Troubleshooting notes cover the common problems.
- "Going further" tasks are optional, for when you finish early.
- If you fall behind, `scripts/catch-up.sh 04` brings your machine to the start of Lab 04.

Your lab machine and United's platform

Each of you has an Ubuntu 24.04 machine with 8 CPUs and 16 GB of memory. You install every tool yourself during the labs.

In the lab	What it does	At United
Harness Open Source	Git hosting, pull requests, pipelines	Harness platform
k3d (Kubernetes in Docker)	Runs containers in a cluster	Amazon EKS
Docker Compose and an ECS task definition file	Runs containers without Kubernetes	Amazon ECS with Fargate
Terraform	Infrastructure as code	AWS CDK shared library
Harness secrets, Kubernetes Secrets	Credentials	TSS

AWS: Amazon Web Services. EKS: Elastic Kubernetes Service. CDK: Cloud Development Kit. TSS: Thycotic One and Secret Server.

DevOps Foundations

Principles, the Three Ways, and delivery performance

KavinSchool

Where DevOps came from

1

2008

Agile Infra

Shafer and Debois meet at Agile 2008 in Toronto to discuss agile infrastructure and cross-silo collaboration.

TORONTO

2

2009

10+ Deploys / Day

Allspaw and Hammond present Flickr's shared tools and trust between dev and operations teams.

VELOCITY CONF

3

2009

devopsdays

Debois organises the first devopsdays in Ghent, Belgium. The name "DevOps" is coined and spreads.

GHENT, BELGIUM

4

2013 – 16

The Literature

The Phoenix Project and The DevOps Handbook codify the principles and Three Ways into clear patterns.

GENE KIM ET AL.

5

2014+

DORA Science

State of DevOps research and DORA prove high throughput directly correlates with system stability.

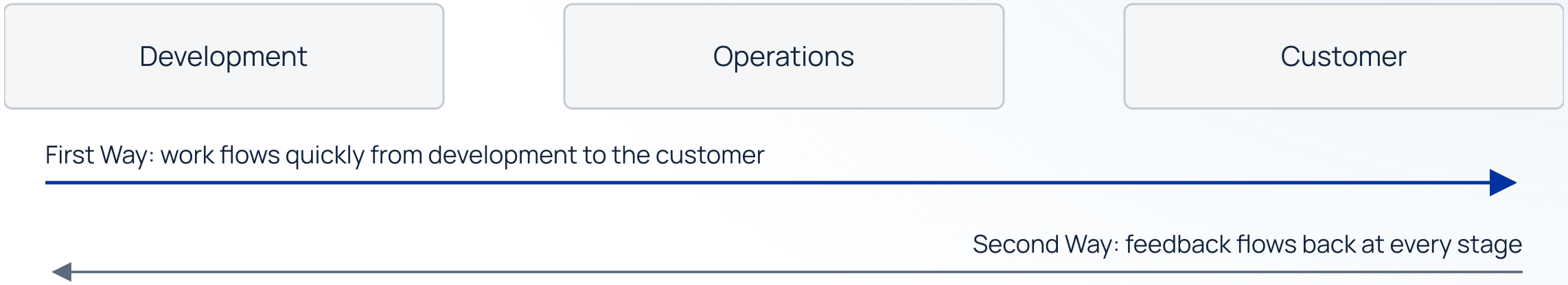
EMPIRICAL SCIENCE

The problem DevOps addresses



The two groups have opposite goals, and work moves between them in large, slow batches. DevOps practices give both groups one shared goal, which is frequent and safe change.

The Three Ways



Third Way: teams experiment, learn from failures, and improve their daily work

The Three Ways are the principles behind most DevOps practices (Kim et al., 2021).

Value streams and lead time



Work time: 6.5 hours

Lead time: about 9 days

Lead time for changes is the time from commit to running in production (Forsgren et al., 2018). In this example the change spends most of its lead time waiting.

Batch size and work in progress

Large Batches

HIGH RISK

- × Long testing and verification cycles
- × Fails more frequently with high blast radius
- × Complex, time-consuming root cause debugging
- × Painful, high-conflict merge sessions

Small Batches

LOW RISK

- ✓ Fast, actionable developer feedback loops
- ✓ Contained blast radius on unexpected failure
- ✓ Trivial to isolate defects and roll back
- ✓ Frictionless, frequent trunk integration

LITTLE'S LAW (OPERATIONS RESEARCH, 1961)

```
Average Lead Time = Work in Progress (WIP) ÷ Throughput
```

Example: 20 items in progress ÷ 5 items finished per week = **4 weeks lead time**.
Takeaway: To shorten lead time without working faster or cutting corners, start fewer things at once.

Measuring delivery performance

Metric	Question it answers	Measures
Deployment frequency	How often do we deploy to production?	Throughput
Lead time for changes	How long does a commit take to reach production?	Throughput
Change failure rate	What share of deployments cause a failure in production?	Stability
Failed deployment recovery time	How long does it take to recover from a failed deployment?	Stability

These are the DORA metrics (Forsgren et al., 2018). The 2024 DORA report adds a fifth measure, rework rate, which is the share of deployments that are unplanned fixes for production problems.

What the research found

FINDING 1

The False Trade-Off

Speed & Stability Go Together

High performers deploy faster and fail less often. Throughput and stability reinforce each other—small, frequent changes are inherently safer.

FINDING 2

Commercial Impact

Directly Predicts Success

Software delivery performance directly drives organizational outcomes: profitability, market share, customer satisfaction, and operational efficiency.

FINDING 3

Causal Drivers

Proven Engineering Practices

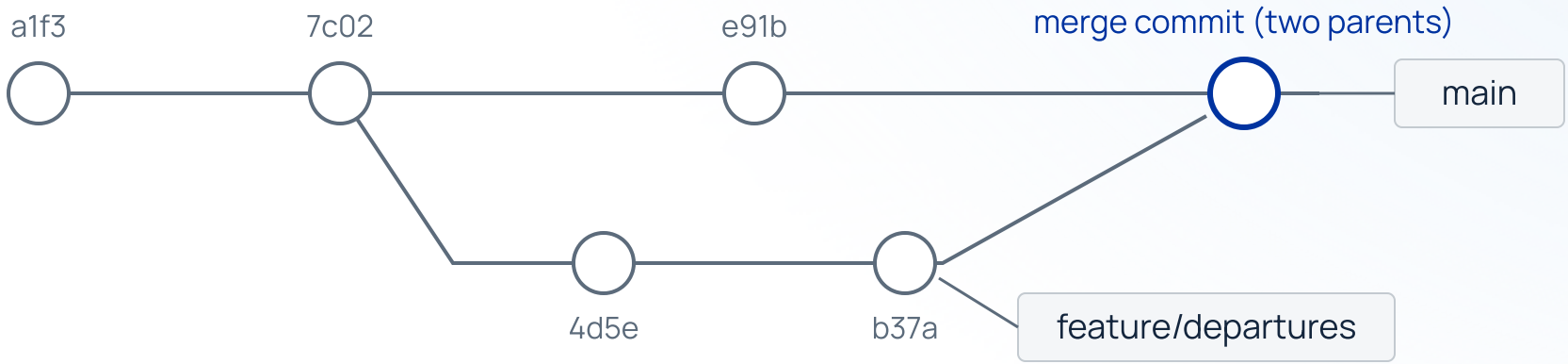
Trunk-based development, continuous integration, comprehensive test automation, and automated deployment causally predict high delivery performance.

Source: Accelerate (Forsgren, Humble & Kim, 2018) — Empirical research across 23,000+ industry respondents.

Version Control and Continuous Integration

Branching workflows, trunk-based delivery, and automated checks

What version control records



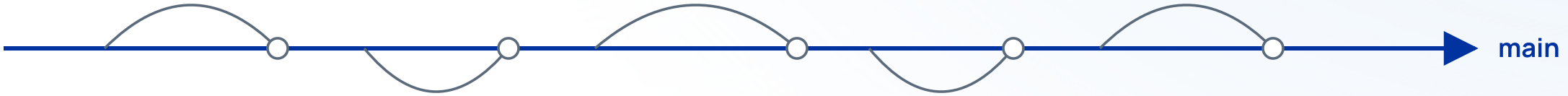
- A commit is a snapshot of all files, plus the author, time, message and parent commit.
- Git names each commit by a hash of its content and its parents. Changing anything changes the hash.
- A branch is a name that points to one commit. A merge commit has two parents.

Two branching strategies

	Git Flow (Driessen, 2010)	Trunk-based development
Long-lived branches	`main`, `develop`, release and hotfix branches	`main` only
Feature branches	Last until the feature is finished, often weeks	Last for hours, at most one or two days
When work is integrated	When a feature branch merges	At least once a day
Releases	Prepared on a release branch	Made from `main` at any time

The labs use trunk-based development with short-lived branches and pull requests.

Trunk-based development



Each branch has one author and lasts for hours, at most one or two days.

- Developers merge small changes into `main` at least once a day.
- Unfinished features are hidden behind feature flags, so `main` can always be released.
- High performers had three or fewer active branches, branches that lived less than a day, and no code freezes (Forsgren et al., 2018).

Pull requests and code review

1. Machine Verification

AUTOMATED CI

- **Automated builds:** compilation & container packaging
- **Static analysis:** linters, formatters, and type checkers
- **Test suites:** unit, API, integration, and coverage checks
- Machines check what machines can objectively prove.

2. Peer Review

HUMAN INSIGHT

- **Architecture & Intent:** Is this the right solution?
- **Design & Readability:** Maintainability, clarity, and edge cases
- **Test quality:** Are we asserting the right business behaviours?

Microsoft Research (Bacchelli & Bird, 2013): Reviews catch far fewer syntax defects than expected. Their primary real-world value is **knowledge transfer** and **shared code ownership**.

The Small PR Golden Rule: Review turnaround speed and critical scrutiny drop precipitously as PR size grows. Keep pull requests scoped to single logical changes readable in under 20 minutes.

Continuous integration practices

- Keep everything needed to build the software in version control.
- Automate the build, and make the build test itself.
- Everyone pushes commits to the main branch every day.
- Every push to the main branch triggers a build.
- Fix a broken build immediately. Keep the build under about ten minutes.

Fowler's article lists more practices. These five are the ones a pipeline enforces (Fowler, 2024).

Press ↓ for supporting detail: What is Continuous Integration?

REFERENCE ONLY

DEEP DIVE / CORE CONCEPT

What is Continuous Integration?

THE PRACTICE

Social Contract

Developers merge working code into the shared main branch daily. Small, frequent merges eliminate painful "merge hell" and expose conflicts while they are still tiny.

THE TOOLING

Automated Gatekeeper

An automated server (like Harness) builds and tests every commit within ten minutes, giving immediate feedback on code health and preventing broken code from reaching others.

KEY INSIGHT

Having a build server is not CI. A team that runs automated builds but merges only once a month is doing automated builds, not Continuous Integration.

Small, frequent changes (General DevOps)

REAL-WORLD SCENARIO

Flickr: development and operations cooperation, Velocity 2009.

- John Allspaw and Paul Hammond described how the photo-sharing service made frequent changes while protecting stability.
- Their approach combined shared version control, automated deployment and shared operational feedback.
- Course connection: make integration routine and inspect feedback while each change is still small.

Source: Flickr engineering, Velocity 2009 slides.

Small, frequent changes (Retail)

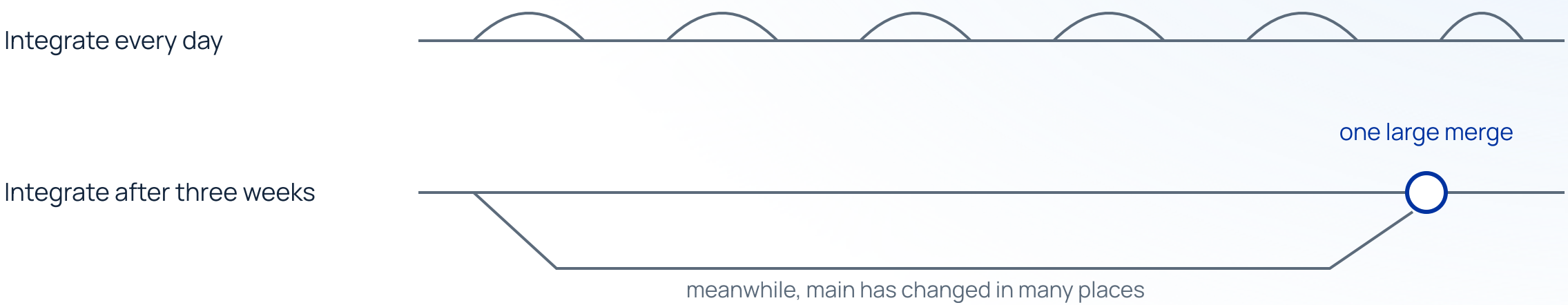
SKIP | REAL-WORLD SCENARIO

Etsy: Deployinator, reported at LISA 2011.

- Etsy described developers deploying production code more than 30 times per day through its deployment tool.
- Small, rapid changes helped the marketplace detect failures and respond quickly.
- Course connection: a short feedback loop makes a failed change easier to identify than a large release batch.

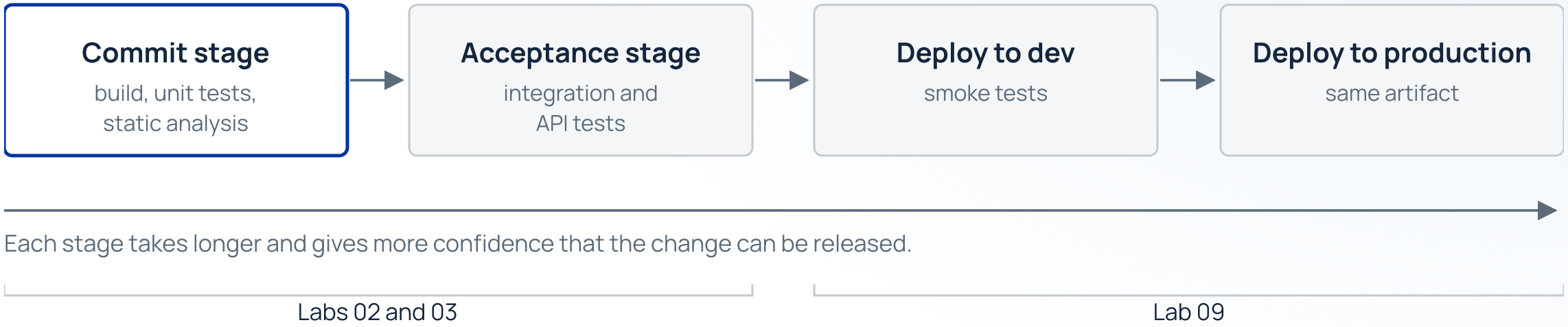
Source: USENIX, Deployinator.

Why integration delay is expensive



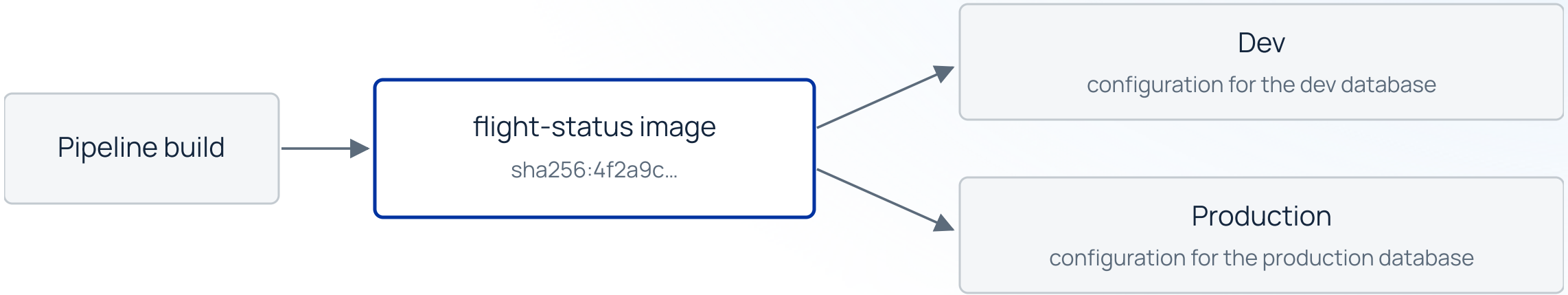
- The longer two branches stay apart, the more both of them change, and the larger the conflicts.
- A conflict found after three weeks involves code that its authors no longer remember well.
- Integrating every day keeps each merge small and each problem easy to locate.

The deployment pipeline



A change moves to the next stage only if it passed the previous one (Humble and Farley, 2010).

Build once, promote the same artifact



- Build the artifact once, in the pipeline. Deploy that same artifact to every environment.
- Only the configuration changes between environments. Keep configuration in the environment (Wiggins, 2011).
- If you rebuild for production, you deploy something that was never tested.

HANDS-ON LAB

Lab 01: Lab Environment and Harness Open Source

Day 1 · 30–40 minutes

HANDS-ON LAB

Lab 01: Lab environment and Harness Open Source

Open `labs/01-lab-environment.md` in the course repository.

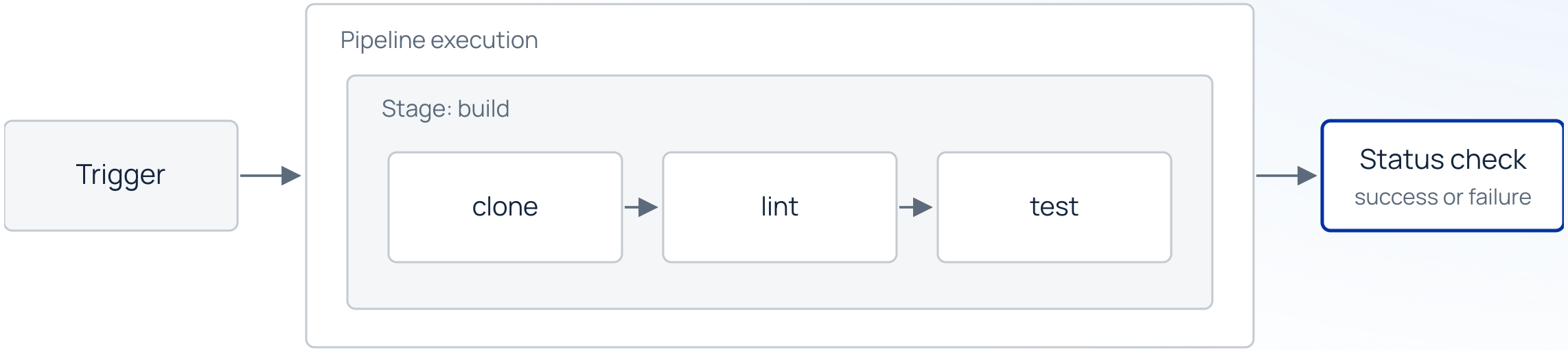
- Install git, curl, jq, Docker and VS Code on your lab machine.
- Start Harness Open Source in a Docker container.
- Create the `flightops` project and the `flight-status` repository.
- Push the starter application and look at it in the Harness web interface.

Pipelines and Harness

Automated build stages, container steps, and triggers

KavinSchool

The parts of a pipeline



- A trigger starts an execution: a push, a pull request, a tag, or a person selecting Run.
- A pipeline has stages. A stage has steps that run in order. A failed step stops the stage.
- The execution reports its status back to the commit and the pull request.

Press ↓ for supporting detail: Hierarchy of a pipeline

REFERENCE ONLY

DEEP DIVE / MENTAL MODEL

Hierarchy of a Pipeline

1. PIPELINE & EXECUTION

Blueprint & Run

Pipeline: The reusable definition in code.
Execution: One specific run for a commit (e.g. build #42).

2. STAGE

The Major Phase

A logical group of steps sharing a purpose (e.g. `ci` stage to build and test; `deploy` stage to release).

3. STEP

The Single Action

An individual command or container task inside a stage (e.g. clone, lint, test). If any step fails, the stage stops immediately.

MANUFACTURING ANALOGY

Think of an assembly line: The pipeline is the factory floor design, the execution is one car passing through, the stages are assembly stations (frame, engine, paint), and the steps are individual operations at each station.

Pipeline as code

```
kind: pipeline
spec:
  stages:
  - name: build
    type: ci
    spec:
      steps:
      - name: test
        type: run
        spec:
          container: python:3.12-slim
          script: pip install -r requirements-dev.txt && pytest
```

- The pipeline is a YAML (YAML Ain't Markup Language) file in the repository: `.harness/ci.yaml`.
- A change to the pipeline is reviewed in a pull request, like a change to the code.
- The history shows who changed the pipeline, when, and why.

Containers as build environments

- Each step names a container image, for example `python:3.12-slim`.
- The image provides the tools. Nothing is installed on the machine that runs the pipeline.
- Every execution starts from the same image, so a result can be repeated.
- Pin the image tag. A floating tag such as `latest` can change between two runs.
- Steps can use different images: Python for tests, Terraform for infrastructure, kubectl for deployments.

Press ↓ for supporting detail: Why run steps in containers?

REFERENCE ONLY

DEEP DIVE / TOOL ISOLATION

Why Run Build Steps in Containers?

THE PAST

Snowflake Build Agents

Compilers and runtimes (Java 8, 11, 17; Python 2/3) installed globally on the build VM. Upgrades for Team A broke Team B's builds. "Passed on agent 1, failed on agent 2."

MODERN CI

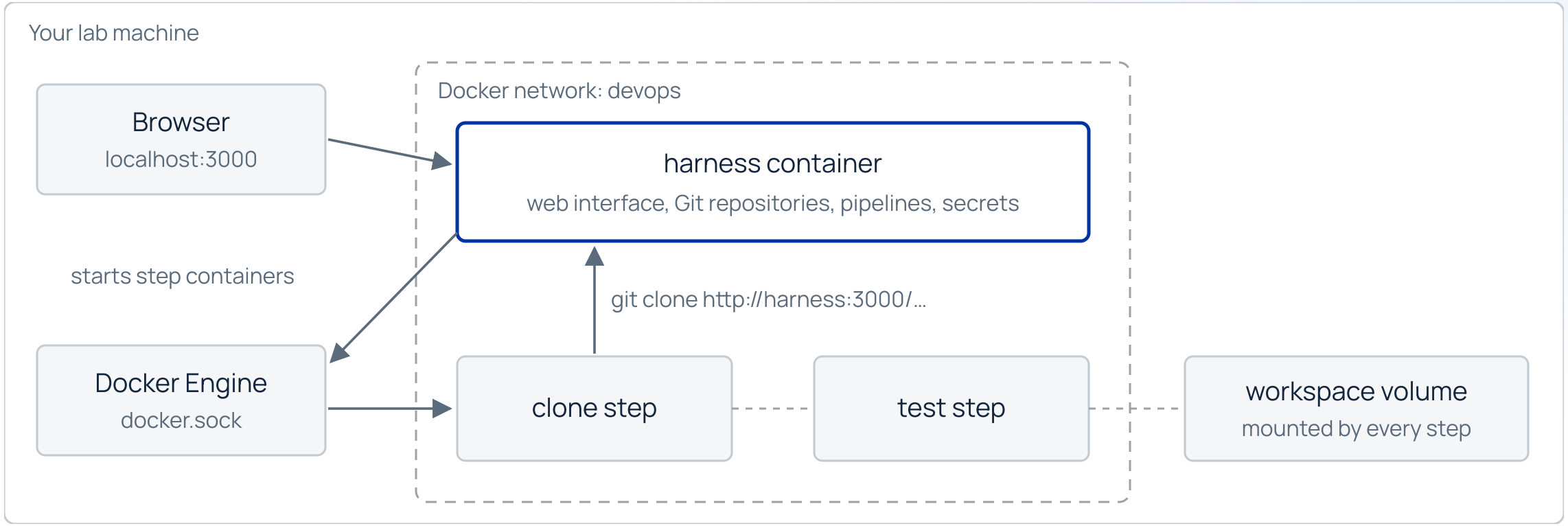
Disposable Toolboxes

Each step pulls an isolated, containerized image (e.g. `python:3.12-slim`). The build host only needs Docker. When the step exits, the container is destroyed without lingering side-effects.

REPRODUCIBILITY RULE

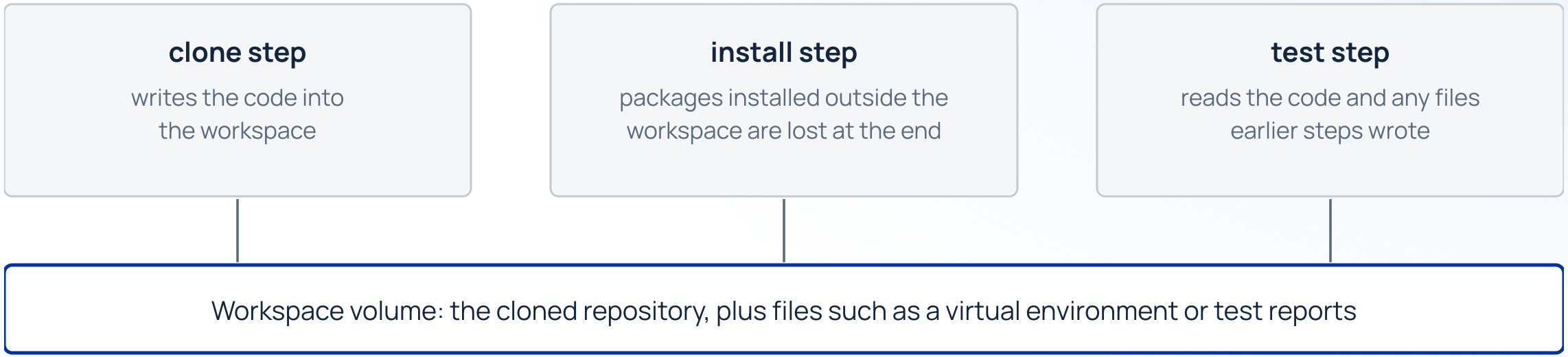
Because the container image is identical on your laptop and in the pipeline, "it works on my machine" problems disappear: you can run the exact same container command locally.

How Harness Open Source runs a step on your machine



Harness asks Docker to start one container per step. The steps reach Harness by the name `harness` on the `devops` network.

Steps share the workspace



- Each step runs in a new container, with its own filesystem.
- Only the workspace is shared. To pass files to a later step, write them into the workspace.

Background services

```
- name: db
  type: background
  spec:
    container: postgres:17-alpine
    envs: {POSTGRES_PASSWORD: test}
- name: integration
  type: run
  spec:
    container: python:3.12-slim
    envs:
      DATABASE_URL: postgresql://postgres:test@db:5432/postgres
    script: pytest -m integration
```

- A background step starts a service and keeps it running while the later steps run.
- Later steps reach the service by the step name as host name: `db`.
- The service stops when the stage ends.

Triggers and events

Event	When it happens	Default trigger
Pull request created or reopened	A pull request is opened or reopened	Yes
Pull request branch updated	New commits are pushed to the branch of an open pull request	Yes
Branch updated	Commits are pushed to a branch, such as `main`	No. Add a trigger.
Tag created	A tag is pushed	No. Add a trigger.
Manual	Someone selects Run	Always available

Lab 02 adds a branch-updated trigger so that every push runs the pipeline, including the merge commits on `main`.

Conditions and expressions

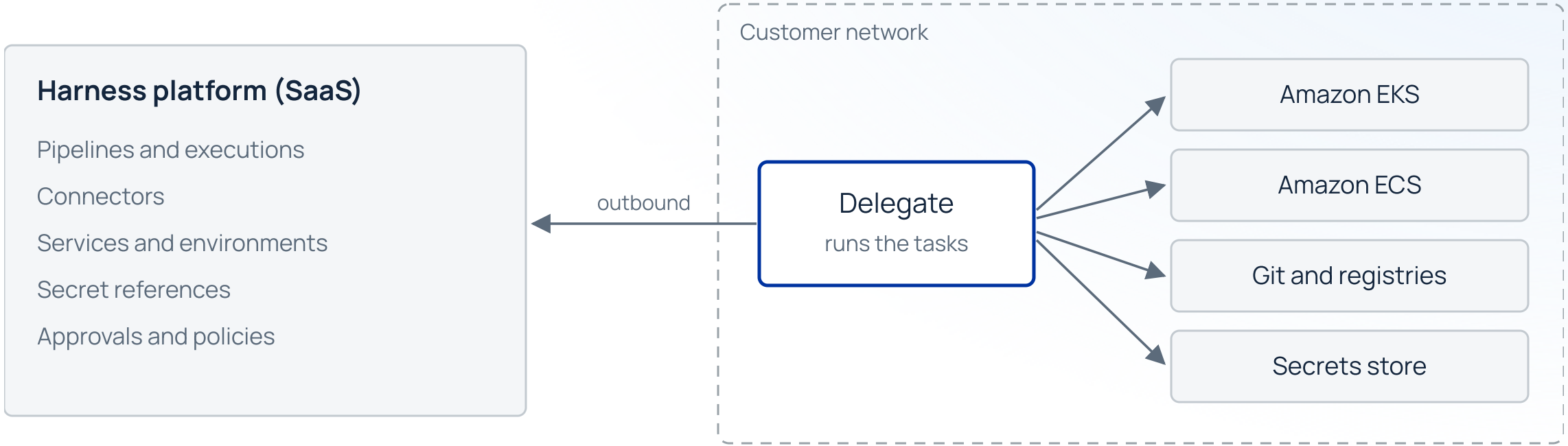
```
- name: report
  type: run
  when: build.event == "pull_request"
  spec:
    container: alpine:3.20
    script: echo "Run ${{ build.number }}; Commit ${{ build.commit }}"
- name: notify
  type: run
  when: failure()
  spec:
    container: alpine:3.20
    script: echo "An earlier step failed"
```

- `${{ … }}` inserts a value when the pipeline runs: the build number, commit, branch or event.
- `when:` decides at run time whether a step runs.
- `failure()` runs a step only after an earlier step failed. `always()` runs it in every case.

Harness Open Source and the Harness platform

Concept	Harness Open Source (lab)	Harness platform (United)
Where steps run	Containers on your lab machine	Harness Cloud machines, or a delegate in the company network
Hierarchy	Project, then repositories	Account, organization, project
Pipeline storage	`.harness/*.yaml` in the repository	In Harness, or in a Git repository
Expressions	`${{ build.number }}`	`<+pipeline.sequenceId>`
Deployments	Scripted in run steps	Deploy stages with services, environments and strategies

Harness platform building blocks



A delegate connects out to Harness and runs tasks inside the network. Connectors tell it how to reach each system.

Press ↓ for supporting detail: How the Delegate protects networks

REFERENCE ONLY

DEEP DIVE / SECURITY ARCHITECTURE

How the Harness Delegate Protects Networks

CONTROL PLANE (SAAS)

Harness Cloud

Hosts the web UI, pipeline graph definitions, audit trails, and execution status. It coordinates work without needing direct inbound access to private cloud resources.

EXECUTION PLANE (PRIVATE)

The Delegate

A lightweight agent running inside the organization's private VPC or Kubernetes cluster. It polls Harness over **outbound HTTPS only**, so no inbound firewall ports are opened.

CONNECTORS & CREDENTIALS

Connectors configure how the Delegate reaches internal targets (Git, clusters, container registries, secret managers). Sensitive credentials stay securely inside your network perimeter.

KNOWLEDGE CHECK

Section Quiz 1: Pipelines in Harness

Twelve questions. Five minutes, including feedback.

- Use the Kahoot game PIN shared by your instructor.
- After each answer, compare the reasoning with your own.

HANDS-ON LAB

Lab 02: Your First CI Pipeline

Day 1 · 30–40 minutes

HANDS-ON LAB

Lab 02: Your first CI pipeline

Open `labs/02-first-ci-pipeline.md`.

- Run the tests in a container on your machine.
- Write `.harness/ci.yaml` with lint, unit test and API test steps, and run it in Harness.
- Read the logs of a failed run and fix the cause.
- Add a trigger so that every push runs the pipeline.

Testing and Quality Gates

The test pyramid, coverage thresholds, and branch rules

The test pyramid



Write many small, fast tests and a few slow, broad ones (Cohn, 2009; Vocke, 2018). Higher layers cost more to write, run and maintain.

Three kinds of tests in the flight-status app

Test	What it checks	What it needs	Time per test
Unit	Rules for flight status changes, input validation	Only the code	Milliseconds
API	HTTP endpoints, status codes and JSON responses	The app with in-memory storage	Tens of milliseconds
Integration	Reading and writing flights in a database	A running Postgres	Up to seconds

Lab 02 runs the unit and API tests. Lab 03 adds the integration tests, with Postgres as a background service.

Test doubles

Double	What it does
Dummy	Fills a parameter that the test does not use.
Stub	Returns fixed answers to calls.
Spy	A stub that also records how it was called.
Mock	Fails the test if the expected calls did not happen.
Fake	A working but simpler implementation, such as an in-memory database.

The flight-status API tests use a fake, an in-memory store, in place of Postgres (Meszaros, 2007).

Flaky tests

DEFINITION A test that **passes and fails on the exact same commit** without any code changes.

1. Timing & Sleep
Fixed timeouts, slow CI runners, async race conditions

2. Shared State
Polluted test DBs, leftover static state, shared caches

3. Test Order
Tests pass when run alone, fail when run in sequence

4. External Calls
Unmocked network APIs, external DNS, time-of-day

Google CI Scale (Micco, 2016)

While only **~1.5%** of test runs report flaky failures, almost **16% of all test cases** exhibit flakiness across their lifecycle—generating thousands of false alarms daily.

The "Cry-Wolf" Trap & Remediation

Flaky tests train developers to ignore failures and re-run pipelines until green. **Golden Rule:** Fix or quarantine (isolate from the blocking gate) immediately to preserve trust in CI.

Reliable test feedback (General DevOps)

REAL-WORLD SCENARIO

Google: continuous testing, reported in 2016.

- Google ran tests before code submission and again after submission to inform release decisions.
- Flaky results made it harder to distinguish a new regression from an unreliable test.
- Course connection: a green rerun does not explain the original failure. Investigate unreliable checks.

Source: Google Testing Blog, Flaky Tests at Google.

Reliable test feedback (Retail)

SKIP | REAL-WORLD SCENARIO

Shopify: improving continuous-integration feedback.

- Shopify reported running more than 170,000 tests in its core monolith and using change-based test selection.
- Its published improvement project reduced the 95th-percentile CI time from 45 minutes to 18.
- Course connection: measure feedback time and reliability before adding more checks to every change.

Source: Shopify, Keeping Developers Happy with a Fast CI.

What code coverage tells you

```
def can_board(status):
    return status in ("SCHEDULED", "DELAYED")

def test_can_board():
    can_board("SCHEDULED")      # runs the line, checks nothing
```

- Coverage measures which lines ran during the tests. This test reaches 100% coverage of `can_board`.
- Coverage does not measure whether the tests check the results.
- In five large Java programs, coverage had a low to moderate correlation with fault detection once the number of tests was held constant (Inozemtseva and Holmes, 2014).
- A coverage threshold is still useful as a floor that blocks large untested changes.

Static analysis and linting

TIER 1: STYLE & QUALITY

Linters & Formatters

e.g. Ruff, Flake8, ESLint

- Reads code syntax without executing it
- Detects dead imports and unused variables
- Enforces consistent team code style
- Eliminates nitpicks in code reviews

< 100ms execution

TIER 2: CORRECTNESS

Type Checkers

e.g. Mypy, TypeScript

- Verifies parameter and return types
- Catches null/None pointer reference bugs
- Enforces interface contracts before runtime
- Protects refactoring across services

< 2s execution

TIER 3: SECURITY

SAST Scanning

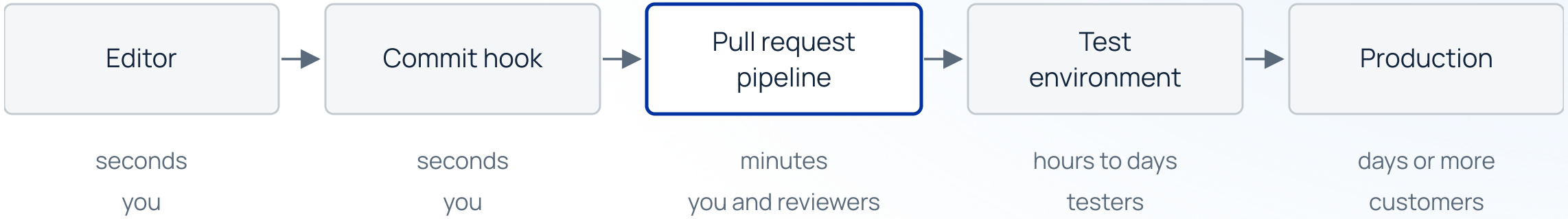
e.g. Bandit, Semgrep, SonarQube

- Flags SQL injection and unsafe string building
- Scans for hardcoded tokens and secrets
- Detects unsafe deserialization & eval()
- Runs before heavy container tests

< 5s execution

Pipeline Optimization Rule: Fastest checks run first. Static analysis completes in seconds without provisioning databases or containers—catching typos and security flaws before expensive integration tests run.

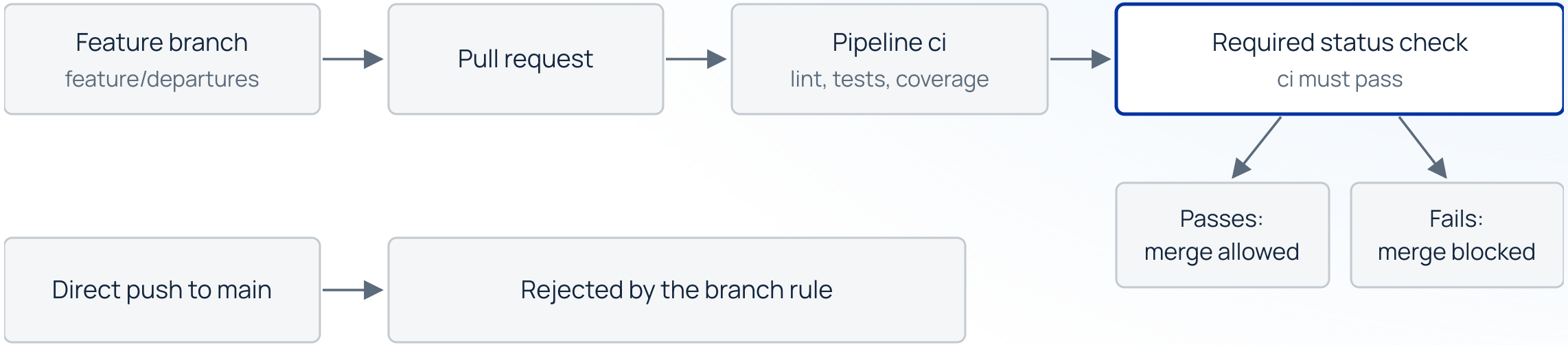
Finding problems earlier



The two lines under each step show how long it takes to find a problem, and who is affected.

"Shift left" means moving checks toward the start of this line. A problem found in the editor costs one person a few seconds. The same problem found in production affects customers.

Quality gates on the main branch



- A branch rule on `main` blocks direct pushes and requires a pull request.
- The rule names the `ci` status check. Merge is possible only after the latest run of `ci` passed.

KNOWLEDGE CHECK

Section Quiz 2: Testing and Quality Gates

Twelve questions. Five minutes, including feedback.

- Use the Kahoot game PIN shared by your instructor.
- After each answer, compare the reasoning with your own.

HANDS-ON LAB

Lab 03: Quality Gates, Pull Requests and Integration Tests

Day 1 · 35–45 minutes

HANDS-ON LAB

Lab 03: Quality gates, pull requests and integration tests

Open `labs/03-quality-gates.md`.

- Add integration tests with Postgres as a background step, and a coverage threshold.
- Protect `main` with a branch rule and a required status check.
- Add a departures board on a branch and open a pull request.
- Fix what the tests and the pipeline find, then merge.

DISCUSSION

Pipeline evidence: Discussion

Explain to a new teammate why a green test run alone does not prove that a change is safe to merge.

1. Think privately for 30 seconds. Name one check and the evidence it provides.
2. In pairs, the explainer has 60 seconds. The listener asks: what could this check miss?
3. Revise one sentence together. Include the check, its evidence and its limit.

Working alone? Write the explanation and challenge it yourself. Sharing aloud is optional.

RECAP

Day 1 summary

MODULE 1

DevOps Foundations

- Fast flow & rapid feedback
- The Three Ways in practice
- DORA metrics: Throughput and Stability reinforce each other

MODULE 2

Trunk & CI

- Trunk-based development
- Daily integration to main
- Small pull requests (< 20 min review)
- Fast 10-minute automated builds

MODULE 3

Pipelines in Harness

- Pipeline as code (YAML)
- Isolated containerized steps
- Automated triggers on PR & push
- Reproducible CI runner environments

MODULE 4

Quality Gates

- Multi-layer test pyramid (Unit, API, DB)
- Sub-second static analysis (Ruff)
- Code coverage enforcement
- Branch protection rules on main

TOMORROW · DAY 2 Infrastructure as Code with Terraform & Secure Credential Management →

References

- Allspaw, J., and Hammond, P. (2009). 10+ Deploys per Day: Dev and Ops Cooperation at Flickr. Velocity Conference.
- Bacchelli, A., and Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. ICSE 2013.
- Chacon, S., and Straub, B. (2014). Pro Git, 2nd ed. Apress. git-scm.com/book
- Cohn, M. (2009). Succeeding with Agile. Addison-Wesley.
- DORA (2024). Accelerate State of DevOps Report. Google Cloud. dora.dev
- Driessen, V. (2010). A successful Git branching model. nvie.com
- Forsgren, N., Humble, J., and Kim, G. (2018). Accelerate. IT Revolution.
- Fowler, M. (2024). Continuous Integration. martinfowler.com/articles/continuousIntegration.html
- Hammant, P. Trunk Based Development. trunkbaseddevelopment.com
- Harness. Harness Open Source and platform documentation. developer.harness.io
- Humble, J., and Farley, D. (2010). Continuous Delivery. Addison-Wesley.
- Inozemtseva, L., and Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. ICSE 2014.
- Kim, G., Humble, J., Debois, P., Willis, J., and Forsgren, N. (2021). The DevOps Handbook, 2nd ed. IT Revolution.
- Little, J. D. C. (1961). A proof for the queuing formula L = λW. Operations Research 9(3).
- Meszaros, G. (2007). xUnit Test Patterns. Addison-Wesley.
- Micco, J. (2016). Flaky Tests at Google and How We Mitigate Them. Google Testing Blog.
- Vocke, H. (2018). The Practical Test Pyramid. martinfowler.com
- Wiggins, A. (2011). The Twelve-Factor App. 12factor.net

Let’s Connect

Scan the QR code or click the link below to stay in touch



Kangeyan Passoubady

linkedin.com/in/kpassoubady

Point your phone camera at the QR code

Thank You

Intermediate DevOps · Day 1: Continuous Integration

KavinSchool